ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ БЮДЖЕТНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
ФИНАНСОВЫЙ УНИВЕРСИТЕТ
ПРИ ПРАВИТЕЛЬСТВЕ РОССИЙСКОЙ ФЕДЕРАЦИИ

АЛТАЙСКИЙ ФИЛИАЛ

Индивидуальный проект

По теме «Разработка веб-сайта с нуля: пошаговая инструкция от идеи до готового ресурса»

Студент _____ Рейник К. С.
(подпись)

Группа 25СП-ИС01

Преподаватель _____ ФИО

Барнаул, 2025

СОДЕРЖАНИЕ

.....1
ВВЕДЕНИЕ.....3
ГЛАВА I.....5
1. ВЕБ-САЙТЫ, ИХ НАПРАВЛЕНИЯ, ОСОБЕННОСТИ, ВИД.....5
1.1 Web-сайты и их направления.....5
1.2 Способы создания сайтов.....6
1.3 Современные направления Веб-дизайна.....7
2. ВНУТРЕННИЙ МИР ВЕБ-САЙТА: РАЗРАБОТКА И ТЕСТИРОВАНИЕ.....8
2.1 Инструменты и среды разработки.....8
2.2 Основы HTML.....9
2.3 Основы CSS.....12
2.4 Хостинг Сайта.....13
ГЛАВА II. СОЗДАНИЕ ИНСТРУКЦИИ И НАПИСАНИЕ САЙТА.....17
1. Создание идеи и макета сайта.....17
2. Создание сайта.....19
3. Выбор хостинга и публикация.....28
ЗАКЛЮЧЕНИЕ.....31
СПИСОК ЛИТЕРАТУРЫ.....32

ВВЕДЕНИЕ

Интернет стал неотъемлемой частью нашей жизни, открывая огромные возможности для общения, работы и творчества. Всё больше людей выходят в сеть, чтобы поделиться информацией, рассказать о себе или своём деле. И умение создавать собственные веб-сайты постепенно превращается из узкоспециального навыка в полезный инструмент для самого обычного пользователя.

Сегодня опубликовать сайт в интернете достаточно просто. Но чтобы он действительно привлекал внимание, важно не только разместить информацию, но и представить её интересно и красиво. Внешний вид и удобство веб-страниц имеют огромное значение — настолько, что их оформление стало отдельным направлением, которое называют веб-дизайном. Поэтому чтобы создать и опубликовать свой сайт, важно разобраться, как и с помощью каких средств это можно сделать.

Проблема: Многие считают, что для того чтобы создать сайт необходимы огромные усилия, навыки и время. Что это делать могут только программисты имеющие, большой опыт за спиной.

Цели работы: Разработать пошаговую инструкция по разработке веб-сайтов. Разработать веб-сайт и выложить его в интернет. Доказать что создание сайтов и его публикация — процессы не требующие огромных усилий и навыков и то что создать сайт может любой человек заинтересованный в этом.

Задачи, которые необходимо выполнить для достижения цели и решения проблемы:

Теоретические задачи:

1. Проанализировать функции, направления и виды web-сайтов
2. Проанализировать актуальные тренды в веб-дизайне
3. Изучить среды разработки, выявить необходимые для создания сайта
4. Освоить необходимые для создания сайта основы языков и методы их использования
5. Проанализировать хостинги

Практические задачи:

1 Планирование

2 Создать прототип (вайрфрейм) блоков и элементов навигации сайта

3 Визуальное проектирование

4 Верстка сайта

5 Наполнение контентом

6 Публикация

ГЛАВА I.

1. ВЕБ-САЙТЫ, ИХ НАПРАВЛЕНИЯ, ОСОБЕННОСТИ, ВИД

1.1 Web-сайты и их направления

Веб-сайт (от англ. *website*: *web* — "паутина, сеть" и *site* — "место") — это совокупность логически связанных веб-страниц, объединённых общим доменным именем (адресом), единым оформлением (дизайном) и навигацией.

По своей сути сайт является сложным информационным продуктом, который существует в цифровом пространстве. Он представляет собой набор страниц объединённых одной идеей, содержанием, дизайном, структурой, функциями и другими элементами в единую систему. Каждый элемент занимает строго свое место и подчиняется общей концепции и архитектуре.

Сайт — многофункциональный инструмент где комбинация возможностей и их реализация определяют конечную ценность и эффективность сайта, поэтому существует множество видов и направлений сайтов которые могут выполнять огромное количество функций и самые различные задачи, но можно выделить их основные функции, направления и виды, которые пользователи встречают чаще всего в интернете:

1. Информационная — предоставление текстовой и графической информации (новости, статьи, описания товаров).
2. Коммуникационная — обеспечение обратной связи через формы, чаты, форумы и комментарии.
3. Коммерческая — продажа товаров и услуг (интернет-магазины, каталоги с возможностью заказа).
4. Сервисная — предоставление полезных инструментов (калькуляторы, переводчики, карты).

5. Развлекательная — видео, игры, музыка и другой досуговый контент.
6. Обучающая — размещение образовательных материалов и курсов.
7 Имиджевая — формирование образа компании или личности (сайты-визитки, портфолио).

Но как правило, сайты редко ограничиваются какими то одними функциями, разработчики постоянно стараются внедрить что то новое для привлечения пользователя.

1.2 Способы создания сайтов.

Также одна из важнейших деталей, определяющих весь дальнейший процесс работы над сайтом, — способ его создания. Именно от этого выбора зависит, потребуется ли разработчику знание кода, какие инструменты будут задействованы, сколько времени уйдёт на реализацию, как в дальнейшем будет обновляться содержимое и насколько легко сайт сможет масштабироваться. Более того, способ создания напрямую влияет на бюджет проекта и требования к хостингу.

1. Статический сайт — это набор готовых страниц которые хранятся на сервере и отправляются браузеру пользователя в неизменном виде в котором были созданы разработчиком.
2. Динамические сайты на системах управления контентом (CMS) — это программное обеспечение, которое позволяет управлять содержимым сайта через удобный интерфейс, не прибегая к написанию кода вручную. написанию кода вручную.
3. Конструкторы сайтов — это онлайн-сервисы, которые позволяют создать полноценный веб-сайт без написания кода, используя визуальный интерфейс перетаскивания блоков (drag-and-drop).

1.3 Современные направления Веб-дизайна.

Смотря на все то что может предложить веб-сайт в плане функций и решения задач поставленных пользователю, ни в коем случае нельзя забывать о внешнем виде и оформлении веб-сайта. Дизайн — это визитная карточка любого сайта и именно благодаря отличным идеям в веб-дизайне и их грамотной реализации можно достичь огромного успеха. Именно хороший и продуманный дизайн заставляет остаться пользователя на сайте. Также как и в реальном мире в среде веб-разработки есть тренды, они задают то как будет выглядеть сайт, то чем он будет наполнен, и какие технологии будут использованы для его реализации.

Основные тренды веб-дизайна в современном мире:

1. Минимализм и скорость

2. Умная анимация

3. Мобильный подход

4. Тёмная тема

5. Крупная типографика

6. Асимметричные макеты

7. 3D-графика и иммерсивность

8. Инклюзивный дизайн

В настоящее время подавляющее большинство качественных сайтов создаётся с соблюдением перечисленных трендов. Следование этим принципам позволяет не только идти в ногу со временем, но и формировать позитивный пользовательский опыт, что напрямую влияет на эффективность ресурса.

2. ВНУТРЕННИЙ МИР ВЕБ-САЙТА: РАЗРАБОТКА И ТЕСТИРОВАНИЕ

2.1 Инструменты и среды разработки.

Выбор инструмента и среды для создания веб-сайта очень важный этап, именно от выбора того где будет создаваться сайт зависит по большей части и качество сайта.

Есть 2 большие группы инструментов для разработки: редакторы кода и интегрированные среды разработки (IDE).

Из множества редакторов кода можно выделить самые популярные и проверенные:

1. VS Code(Visual Studio code) — Самый популярный инструмент среди веб-разработчиков. Был создан компанией Microsoft и является бесплатным, работает на всех системах и может быть использован для любых задач благодаря огромному количеству расширений.
2. Другим популярным решением является Sublime Text. Этот редактор ценится за свою невероятную скорость работы. Он моментально открывает даже очень большие файлы и потребляет минимум ресурсов.
3. Notepad++ — лёгкий редактор для Windows. Часто используется в образовательных учреждениях на начальном этапе изучения HTML и CSS.
4. WebStorm — профессиональная IDE от компании JetBrains. Это мощный инструмент, который «из коробки» содержит всё необходимое для веб-разработки.
5. Особняком стоят онлайн-редакторы и «песочницы» — сервисы, которые позволяют писать код прямо в браузере и сразу видеть результат. CodePen, JSFiddle, Replit — яркие представители этого класса.

Помимо редактора кода, веб-разработчику необходимы и другие инструменты:

1. Браузер с инструментами разработчика (DevTools). Любой современный браузер (Chrome, Firefox, Edge) имеет встроенный набор инструментов для разработки, вызываемый клавишей F12.
2. Инструменты веб-дизайна — Figma, Krita. GIMP. Очень удобные и мощные инструменты для дизайна.
3. Система контроля версий Git. Git отслеживает все изменения в проекте и позволяет вернуться к любой предыдущей версии файлов. В сочетании с платформой GitHub это даёт не только удобное хранилище кода, но и возможность бесплатного хостинга через GitHub Pages.

Выбор инструмента напрямую зависит от поставленных задач. Но для первого и несложного сайта хватит простого редактора кода как VS Code, графического редактора можно использовать Krita, и браузера с инструментами разроботчика.

2.2 Основы HTML.

HTML (HyperText Markup Language — язык гипертекстовой разметки) — это фундамент любого веб-сайта. Без него не может существовать ни одна веб-страница.

HTML не является языком программирования. Он не умеет выполнять вычисления, принимать решения или обрабатывать данные. Его задача — структурировать информацию: разбить текст на заголовки, абзацы, списки; вставить изображения, таблицы, ссылки; выделить смысловые блоки. HTML говорит браузеру: «Вот это — заголовок, это — навигационное меню, а это — подвал страницы».

HTML-документ представляет собой обычный текстовый файл с расширением .html, который состоит из элементов — тегов. Теги заключаются в угловые скобки и чаще всего бывают парными: открывающий <тег> и закрывающий </тег>. Внутри между ними находится содержимое, а сам тег указывает, как это содержимое нужно интерпретировать.

Пример тега:

Это абзац текста

Здесь - открывающий тег — начало абзаца. - закрывающий тег (конец абзаца

Так же есть теги которым являются одиночными и им не нужен закрывающий тег к примеру: <meta>

Все теги имеют важное значение и знать хотя бы большую часть из них очень важно.

Список основных тегов используемых при создании сайтов:

1. Основная структура документа.

<html>	Корневой элемент страницы. Контейнер, внутри которого находится весь HTML-код.
<head>	Служебная часть страницы. Содержит мета-информацию, заголовок вкладки, подключение стилей и скриптов. Не отображается на экране.
<body>	Тело страницы. Всё, что находится внутри, видит пользователь: тексты, картинки, кнопки, меню.
<!DOCTYPE html>	Не тег, а инструкция для браузера. Указывает, что используется стандарт HTML5.

2. Служебные теги(внутри тега <head>).

<title>	Заголовок вкладки браузера. Размещается только внутри <head>.
<meta>	Одиночный тег. Задаёт служебную информацию: кодировку, описание страницы, ключевые слова, настройки для мобильных устройств.
<link>	Подключение внешних файлов (CSS-стили, шрифты, иконки). Размещается в <head>.
<style>	Контейнер для внутренних CSS-стилей. Размещается в <head>
<script>	Подключение JavaScript-кода или ссылка на внешний скрипт. Может

	размещаться в <head> или перед закрывающим </body>.

3. Семантические теги структуры

<header>	Шапка сайта или раздела. Обычно содержит логотип, название, навигацию.
<nav>	Навигационное меню. Содержит ссылки на разделы сайта.
<main>	Основное уникальное содержимое страницы. Должен быть один на странице.
<section>	Логический раздел страницы. Обычно имеет заголовок.
<article>	Самостоятельная законченная часть: статья, новость, пост, карточка товара.
<aside>	Боковая колонка, дополнительная информация, сайдбар.
<footer>	Подвал страницы или раздела. Контакты, копирайт, ссылки на политику.

4. Заголовки и текст.

<h1>-<h6>	Заголовки шести уровней. `<h1>` — самый важный (один на страницу), <h6> — самый мелкий.
<p>	Абзац текста.
 	Одиночный тег. Перенос строки без создания нового абзаца.
<hr>	Одиночный тег. Горизонтальная разделительная линия.
	Жирное начертание. Обозначает важность текста логическое ударение).

5. Ссылки и изображения.

<a>	Ссылка. Атрибут href задаёт адрес перехода.
<img>	Одиночный тег. Вставка изображения. Обязательные атрибуты: src (путь), alt (описание).

Ошибки в структуре могут быть незаметны новичку, но браузер их чувствует. Например, если забыть закрыть тег </body>, современный браузер чаще всего

всё равно отобразит страницу. Но если в большом проекте пропустить закрывающий тег или перепутать вложенность — всё поедет: стили перестанут применяться, скрипты сломаются, а верстальщик, который придёт после автора, потратит часы на поиск одной пропущенной скобки.

2.3 Основы CSS.

Если HTML — это фундамент и несущие стены сайта, то CSS (Cascading Style Sheets — каскадные таблицы стилей) — это его внешний вид: цвет стен, форма окон, обои, шрифт вывески и расстояние между предметами мебели. Без CSS страница выглядит как сплошной чёрно-белый текст с картинками, выстроенными в одну колонку. Именно CSS превращает «документ» в «дизайн».

CSS не является языком программирования. Как и HTML, это язык описания. Он не умеет вычислять, принимать решения или обрабатывать данные.

Перед тем как начать стилизовать страницу, надо подключить файл. Стили выносятся в отдельный файл с расширением .css и подключается через тег <link> в <head>

```
<link rel="stylesheet" href="style.css">
```

Его преимущество в том что один файл стилей может управлять сотнями страниц. Изменил цвет в одном месте — поменялось везде.

Синтаксис CSS

CSS-правило состоит из трех частей:

```
селектор {
    свойство: значение;
    свойство: значение;
}
```

Селектор — указывает, к каким элементам применить стили.

Свойство — что именно меняем (цвет, размер, отступ).

Значение — как меняем (красный, 20px, сверху).

Часто используемые CSS-свойства

color	Цвет текста	background	Цвет фона
font-size	Размер шрифта	font-weight	Жирность шрифта
font-family	Шрифт	text-align	Выравнивание текста
text-decoration	Подчеркивание	line-height	Межстрочный интервал
width	Ширина элемента	height	Высота элемента
max-width	Максимальная ширина	margin	Внешний отступ
padding	Внутренний отступ	border	Рамка
border-radius	Скругление углов	display	Тип отображения
flex	Управление во Flexbox	gap	Отступы между элементами
position	Позиционирование	box-shadow	Тень

Освоив базовые селекторы, блочную модель, Flexbox и адаптивность, можно сверстать любой макет средней сложности. Остальное — дело практики и насмотренности.

2.4 Хостинг Сайта

После создания сайта, его нужно сделать общедоступным в интернете. Для этого прибегают к хостингам.

Хостинг — это место на сервере, где хранятся все файлы сайта: HTML-страницы, CSS-стили, изображения, скрипты. Хостинг-провайдеры предоставляют такие серверы в аренду. Серверы работают круглосуточно, чтобы сайт был доступен в любое время.

Домен — это адрес сайта в интернете, который пользователи вводят в строку браузера (например, mysite.ru). Домены регистрируются на определенный срок (обычно на год) и требуют продления. Без домена сайт доступен только по IP-адресу сервера, что неудобно для пользователей.

Существует несколько видов хостинга, которые различаются по возможностям, стоимости и сложности настройки.

Бесплатный хостинг
Подходит для учебных проектов, тестирования и небольших статических сайтов. Основные особенности:

1. Не требует финансовых затрат
2. Часто имеет ограничения по месту и функционалу
3. Может содержать рекламу хостинг-провайдера
4. Домен обычно имеет вид username.provider.com (свой домен подключить сложно или платно)

Виртуальный хостинг (shared hosting)
Самый популярный вариант для небольших и средних сайтов. Один физический сервер делится между множеством пользователей. Особенности:

1. Низкая стоимость (от 100–200 рублей в месяц)
2. Простая настройка через панель управления
3. Подходит для большинства сайтов (WordPress, интернет-магазины)
4. Ресурсы (процессор, память) делятся между соседними сайтами

Выделенный сервер (VPS/VDS)
Виртуальный сервер, который выделяется в распоряжение одного клиента. Особенности:

1. Гарантированные ресурсы (процессор, память, дисковое пространство)
2. Полный доступ к настройкам сервера
3. Подходит для проектов с высокой нагрузкой (крупные магазины, порталы)
4. Выше стоимость (от 300–500 рублей в месяц)

Выделенный физический сервер (dedicated server)
Целый физический сервер в распоряжении одного клиента. Используется для очень крупных проектов с высокой посещаемостью и особыми требованиями к безопасности.

Для небольших статичных сайтов хватит бесплатного хостинга на типе GitHub Pages или Netlify.

Для сайтов на CMS (WordPress, Joomla) лучше использовать виртуальный хостинг, к примеру TimeWeb, REG.RU или Beget.

Для более масштабных сайтов, например крупный интернет магазин используют VPS/VDS сервера. Они очень хороши для таких сайтов, можно самому подобрать необходимые характеристики сервера.

Для крупных т сразу выделенные. Они обеспечивают максимальную производительность и позволяют держать на себе огромные сайты.

Процесс публикации сайта на хостинге включает несколько этапов:

Шаг 1. Выбор хостинг-провайдера и тарифа
Определяется, какой хостинг подходит для проекта (бесплатный, виртуальный, VPS). Заключается договор или регистрируется аккаунт.

Шаг 2. Регистрация домена (если нужен)
Домен можно зарегистрировать у того же провайдера или отдельно. Если домен регистрируется отдельно, его нужно привязать к хостингу через DNS-настройки.

Шаг 3. Загрузка файлов сайта
Все файлы сайта (HTML, CSS, JS, изображения) загружаются на сервер хостинга.

Шаг 4. Настройка домена
Если домен регистрировался отдельно, нужно указать DNS-серверы хостинга в настройках домена. Если домен регистрировался у того же провайдера, настройки часто происходят автоматически.

Шаг 5. Проверка работы сайта
После загрузки файлов и настройки домена сайт становится доступен по адресу. Необходимо проверить, что все страницы открываются, изображения загружаются, ссылки работают.

сайтов и сервисов по типу VK, Rutube, Telegram, Ozon и многих других берут сразу выделенные. Они обеспечивают максимальную производительность и позволяют держать на себе огромные сайты.

Процесс публикации сайта на хостинге включает несколько этапов:

Шаг 1. Выбор хостинг-провайдера и тарифа
Определяется, какой хостинг подходит для проекта (бесплатный, виртуальный, VPS). Заключается договор или регистрируется аккаунт.

Шаг 2. Регистрация домена (если нужен)
Домен можно зарегистрировать у того же провайдера или отдельно. Если домен регистрируется отдельно, его нужно привязать к хостингу через DNS-настройки.

Шаг 3. Загрузка файлов сайта
Все файлы сайта (HTML, CSS, JS, изображения) загружаются на сервер хостинга.

Шаг 4. Настройка домена
Если домен регистрировался отдельно, нужно указать DNS-серверы хостинга в настройках домена. Если домен регистрировался у того же провайдера, настройки часто происходят автоматически.

Шаг 5. Проверка работы сайта
После загрузки файлов и настройки домена сайт становится доступен по адресу. Необходимо проверить, что все страницы открываются, изображения загружаются, ссылки работают.

Главное правило — хостинг должен соответствовать типу сайта: статическому сайту не нужна база данных и серверный код, поэтому он отлично работает на бесплатном хостинге. А крупным проектам с тысячами посетителей требуются мощные серверы с гарантированными ресурсами. Правильный выбор хостинга обеспечивает стабильную работу сайта и комфорт для его посетителей.

ГЛАВА II. СОЗДАНИЕ ИНСТРУКЦИИ И НАПИСАНИЕ САЙТА

В данной главе описывается практический процесс создания сайта. Вся вторая глава это пошаговая инструкция, которую может повторить любой желающий.

1. Создание идеи и макета сайта

1.1 Планирование

Перед тем как начать писать сайт сайт надо все спланировать.

1. Определить тему сайта и его содержание — нужно четко понимать, о чем будет сайт, какую информацию он будет нести, какие разделы будут на нем представлены.
2. Определить целевую аудиторию — люди, которым будет интересно то, что представлено на сайте. От этого зависят стиль оформления, тон текстов и выбор функций.
3. Поставить цели сайта — определить, зачем он нужен. Цели могут быть разными: рассказать о проекте, привлечь новых участников, дать возможность связаться с автором, поделиться результатами работы.
4. Продумать структуру — то, как будет выглядеть сайт: какие страницы будут на нем, как они будут связаны между собой, что будет на каждой странице.
5. Составить план контента — подготовить список текстов, изображений и другого наполнения, которое потребуется для каждой страницы.

В случае с сайтом

1. Тема сайта — индивидуальный проект «Разработка веб-сайта с нуля: пошаговая инструкция от идеи до готового ресурса». Сайт рассказывает о создании проекта и содержит готовый результат.
2. Целевая аудитория — студенты, начинающие разработчики, преподаватели и все, кто интересуется веб-разработкой.
3. Цели сайта — представить результат проекта, рассказать о процессе создания, дать пример для обучения, показать доступность веб-разработки для новичков.
4. Структура сайта — 5 страниц:
5. План контента — подготовлены тексты для всех страниц, логотип, скриншоты кода и PDF-документ проекта.

Страница	Содержание
Главная	Приветствие, краткое представление
О проекте	Описание целей и задач
Проект	PDF-версия работы
Сайт	Информация о создании, примеры кода
Обо мне	Информация об авторе

Результат: четкий план, который стал основой для всех последующих этапов — от прототипирования до публикации.

1.2 Создание прототипа и визуальное проектирование

Прототип — схема будущего сайта, которая показывает расположение основных блоков и элементов навигации без детальной проработки дизайна.

В случае с сайтом схема была нарисована в программе Krita. На ней сначала было указано то какой вид будут иметь все страницы изначально — неизменяемые блоки

1. header (шапка страницы) — на ней кнопка для перехода на главную страницу, логотип и заголовок текущей страницы.
2. nav (навигационное меню) — здесь ссылки на другие страницы: «О проекте», «Проект», «Сайт», «Обо мне».
3. footer (подвал) — Копирайт

Следом идет вид блока с основным контентом (main)

1. На странице «О проекте» указан краткий текст о том зачем и на что ориентирован этот проект.
2. На странице «Проект» указано место для самого проекта который можно просмотреть в самом сайте.
3. На странице «Сайт» указаны этапы создания от создания прототипа до готового сайта с приложенными к этому скриншотами работы и файлами сайта.
4. На странице «Обо мне» указага краткую информацию о создателе сайта и проекта.

После создания макета идет визуальное проектирование, созданию дизайна и конечного вида сайта.

Подбор цветов. Было решено использовать темные тона в шапке и подвале для создания контраста с основным контентом.

1. Фон шапки в темно-сером цвете.
2. Блоки в навигации — градиентная шкала в синих цветах
3. Подвал: темно-серый
4. Основной фон: белый

Выбор шрифтов для создания более приятного и интересного стиля. Шрифты были взяты из Google Fonts:

Oswald — для основного текста и навигации.

PT sans — для заголовков в шапке.

Как результат получен готовый дизайн — макет, по которому выполнялась дальнейшая верстка.

2. Создание сайта

2.1 Верстка сайта

После формирования идеи и создания дизайна, следующее это написание сайта на языках программирования.

Первым делом это создание структуры сайта. Для этого используются семантические теги основные из них: <body>, <header>, <nav>, <main>, <aside>, <footer>.

Для построения сайта важно сохранять вложенность и иерархию:

В теге Body размещаем все остальные

```
<body>
    <header>
    </header>
    <nav>
    </nav>
    <main>
        <section>
        </section>
        <article>
        </article>
    </main>
```

```
    <aside>
    </aside>
    <footer>
    </footer>
</body>
```

Правила вложенности

1. Тег <body> — это корневой контейнер видимой части страницы. Все остальные элементы, которые пользователь видит на экране, должны находиться внутри него. Нельзя размещать элементы вне <body>.

2. Тег <main> — может быть только один на странице. Он содержит уникальное содержимое, которое не повторяется на других страницах. Нельзя вкладывать <main> в <header>, <footer> или <aside>.

3. Теги <header> и <footer> — могут использоваться как для всей страницы, так и внутри разделов. Например, внутри <article> может быть свой <header> с заголовком и датой публикации.

4. Тег <section> — используется для группировки связанного контента. Обычно внутри него есть заголовок (<h2> или <h3>). Не используйте <section> вместо для простого оформления.

5. Тег <article> — предназначен для самостоятельного, независимого контента: новость, пост, карточка товара. Может содержать внутри свой <header> и <footer>.

6. Тег <aside> — используется для дополнительной информации, которая связана с основным контентом, но не является его частью: боковая колонка, виджеты, реклама.

В случае с моим сайтом я повторил структуру написанную мною ранее и перешел к наполнению сайта контентом.

Основными тегами в наполнении послужили

Для текста и ссылок

1. <h1>, <h2>, <h3> - заголовки

   <p> - абзацы

   <a href=""></a> - ссылки
2. Для изображений

<img src="" alt=""> - вставка изображения

3. Для остального наполнения

   <embed src=""/> - встраивание внешних документов

Блоки-контейнеры (div) и их классы

Для удобства стилизации и структурирования контента я использовал универсальные блоки с классами. Классы позволяют обращаться к этим блокам из CSS и задавать им стили, а также делают код понятным для других разработчиков.

Основные классы, которые я использовал в HTML:

Класс	Назначение	Где используется
.header-container	Контейнер для содержимого шапки	Внутри <header>
.header-left	Левая часть шапки	Содержит логотип и кнопку «На главную»
.header-right	Правая часть шапки	Пустой блок для симметрии
.site-logo	Стилизация логотипа	Тег <img> с логотипом
.home-link	Стилизация кнопки «На главную»	Ссылка <a>
.content-block	Блок с текстом и изображением	Внутри <main>
.text	Блок с текстовым содержимым	Внутри .content-block
.image	Блок с изображением	Внутри .content-block
.reverse	Перевернутое расположение блоков	Дополнительный класс для .content-block
.pdf-viewer	Контейнер для PDF-просмотрщика	На странице «Проект»

Использование блоков с классами в HTML позволило:

1. Создать единую структуру для всех страниц
2. Упростить управление расположением элементов через CSS
3. Легко вносить изменения в дизайн
4. Поддерживать читаемость кода

В сочетании с семантическими тегами (<header>, <nav>, <main>, <footer>) и классами для стилизации, структура сайта получилась логичной, удобной для поддержки и легко расширяемой.

После создания HTML-структуры я перешел к стилизации — заданию внешнего вида страниц через CSS. Ниже представлена пошаговая инструкция по настройке стилей для сайта.

Шаг 1. Сброс отступов

Первым делом необходимо обнулить стандартные отступы браузеров, чтобы сайт выглядел одинаково во всех браузерах.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

margin: 0 и padding: 0 — убирают лишние отступы

box-sizing: border-box — ширина элемента включает padding и border

Шаг 2. Базовые настройки body

Задаем основные стили для всей страницы: шрифт, фон, прижатие подвала.

```
body {
    display: flex;
    flex-direction: column;
    min-height: 100vh;
    font-family: 'Oswald', sans-serif;
}
```

display: flex и flex-direction: column — позволяет прижать подвал к низу

min-height: 100vh — высота страницы не меньше высоты экрана

font-family — основной шрифт сайта

Шаг 3. Стилизация шапки

Шапка сайта должна быть темной, чтобы выделяться на фоне контента. Логотип и кнопка располагаются слева, заголовок — по центру.

```
header {
    background-color: #2b2b2b;
    padding: 10px 0;
}
.header-container {
    max-width: 1400px;
    margin: 0 auto;
    display: flex;
    align-items: center;
    justify-content: space-between;
}
.header-left, .header-right {
    flex: 1;
    display: flex;
    gap: 15px;
}
.header-left {
    justify-content: flex-start;
}
.header-right {
    justify-content: flex-end;
}
header h2 {
    font-size: 3rem;
    color: white;
    text-shadow: 5px 5px #333;
```

```
}
```

.header-container — ограничивает ширину и центрирует содержимое

.header-left и .header-right с flex: 1 — обеспечивают центрирование заголовка

text-shadow — улучшает читаемость текста на темном фоне

Шаг 4. Стилизация кнопки «На главную»

Кнопка должна быть заметной, но не перетягивать внимание. При наведении — плавная анимация.

```
.home-link {
    color: white;
    background-color: rgba(255, 255, 255, 0.1);
    padding: 8px 15px;
    border-radius: 30px;
    transition: 0.3s;
}
.home-link:hover {
    background-color: rgba(255, 255, 255, 0.2);
    transform: translateY(-2px);
}
```

transition — обеспечивает плавность анимации

transform: translateY(-2px) — кнопка поднимается при наведении

Шаг 5. Стилизация навигационного меню

Меню должно занимать всю ширину, пункты — равномерно распределяться. Каждый пункт имеет свой цвет фона.

```
nav {
    display: flex;
    width: 100%;
}
nav a {
```

```
    flex: 1;
    font-size: 1.5rem;
    color: white;
    text-align: center;
    padding: 15px 0;
    transition: 0.3s;
}
nav a:nth-child(1) { background-color: #0a2f6a; }
nav a:nth-child(2) { background-color: #1a4b8c; }
nav a:nth-child(3) { background-color: #2a6bae; }
nav a:nth-child(4) { background-color: #3a8fd0; }

nav a:hover {
    opacity: 0.8;
    transform: translateY(-2px);
}
```

flex: 1 — все пункты одинаковой ширины

:nth-child() — задает цвет каждому пункту по порядку

opacity — при наведении пункт становится полупрозрачным

Шаг 6. Стилизация основного контента

Блоки контента состоят из текста и изображения, расположенных рядом. Заголовки выделены синим цветом.

```
main {
    flex: 1;
    max-width: 1200px;
    margin: 30px auto;
    padding: 0 20px;
}
```

```
.content-block {
    display: flex;
    align-items: center;
    gap: 40px;
    margin-bottom: 60px;
}

.text, .image {
    flex: 1;
}

.text h2, .text h3 {
    color: #0a2f6a;
    margin-bottom: 20px;
}

.text p {
    line-height: 1.6;
    color: #333;
}

.image img {
    width: 100%;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}
```

flex: 1 — текст и изображение занимают равную ширину

gap: 40px — расстояние между текстом и изображением

border-radius и box-shadow — делают изображения аккуратнее

Шаг 7. Стилизация подвала

Подвал — светлый, с копирайтом по центру.

```
footer {
    background-color: #f0f0f0;
    padding: 20px;
    text-align: center;
}
```

Шаг 8. Адаптивная верстка

Для мобильных устройств (ширина экрана менее 768px) нужно перестроить расположение элементов.

```
@media (max-width: 768px) {
    .header-container {
        flex-direction: column;
    }
    .header-left {
        width: 100%;
        justify-content: center;
    }
    .header-right {
        display: none;
    }
    header h2 {
        font-size: 2.5rem;
    }
    nav {
```

```
        flex-direction: column;
    }

    .content-block {
        flex-direction: column;
    }
}
```

flex-direction: column — элементы выстраиваются вертикально

display: none — скрывает правый блок шапки на телефонах

Размер шрифта заголовка уменьшен

Результат

Стилизация через CSS обеспечила:

1. Единый визуальный стиль — все страницы выглядят одинаково
2. Адаптивность — сайт корректно отображается на телефонах и планшетах
3. Плавные анимации — кнопки и ссылки реагируют на наведение
4. Читаемость — правильные отступы, контрастные цвета, тени

3. Выбор хостинга и публикация

Так как сайт статический было решено использовать бесплатный хостинг GitHub Pages.

Для создания необходимо:

1. Зарегистрировал аккаунт на GitHub
2. Создал новый репозиторий (хранилище для файлов)
3. Назвал репозиторий в соответствии с проектом
4. Установил репозиторий как публичный

После загрузки всех файлов надо активировать.

1. Переходим в настройки репозитория (Settings)
2. Далее в раздел Pages

3. В поле Branch выбрать ветку main
4. Нажать Save

Через несколько минут сайт должен быть доступен по адресу

https://username.github.io/repository-name/
После публикации необходимо проверить:

1. Главная страница — открывается, все элементы отображаются
2. Навигация — все ссылки ведут на нужные страницы
3. Изображения — загружаются и отображаются корректно
4. PDF-документ — открывается в просмотрщике
5. Адаптивность — на телефоне сайт выглядит правильно
6. Скорость загрузки — страницы открываются быстро

В результате публикации:

1. Сайт стал доступен в интернете по постоянному адресу
2. Все 5 страниц работают корректно
3. Сайт открывается на компьютерах, планшетах и телефонах
4. При обновлении файлов достаточно загрузить их в репозиторий — сайт обновится автоматически

В ходе выполнения индивидуального проекта на тему «Разработка веб-сайта с нуля: пошаговая инструкция от идеи до готового ресурса» была достигнута поставленная цель — разработана пошаговая инструкция по созданию веб-сайта и создан готовый ресурс, опубликованный в интернете.

В процессе работы были решены все поставленные задачи:

Теоретические задачи:

1. Проанализированы функции, направления и виды веб-сайтов. Выделены основные типы: информационные, коммерческие, сервисные. Рассмотрены сайты-визитки, портфолио, лендинги, блоги, каталоги и проектные сайты.
2. Изучены актуальные тренды веб-дизайна: минимализм, мобильный подход, крупная типографика, темная тема, асимметричные макеты, инклюзивный дизайн.

3. Рассмотрены среды разработки: редакторы кода (VS Code, Sublime Text, Notepad++) и графические редакторы (Figma, GIMP, Krita).
4. Освоены основы языков HTML и CSS — необходимые для создания статического сайта.
5. Изучены методы тестирования: проверка в браузерах, адаптивность, скорость загрузки.
6. Проанализированы хостинги для размещения сайтов: бесплатные (GitHub Pages, Netlify) и платные (Beget, Timeweb).

Практические задачи:

1. Проведено планирование: определена тема, целевая аудитория, цели и структура сайта.
2. Создан прототип (вайрфрейм) в Krita, показывающий расположение основных блоков и навигации.
3. Выполнено визуальное проектирование: разработан дизайн-макет, подобраны цвета и шрифты.
4. Выполнена верстка сайта с использованием семантических тегов HTML и каскадных таблиц стилей CSS. Создано 5 страниц с единой структурой.
5. Проведено наполнение контентом: добавлены тексты, изображения, PDF-документ проекта.
6. Осуществлена публикация сайта на бесплатном хостинге GitHub Pages.

ЗАКЛЮЧЕНИЕ

В результате работы был создан сайт, состоящий из 5 страниц: главная страница, страница «О проекте», страница «Проект» с PDF-просмотрщиком, страница «Сайт» с информацией о создании ресурса, страница «Обо мне». Сайт адаптирован для всех устройств, имеет семантически правильную структуру, прошел тестирование и успешно опубликован по адресу: https://username.github.io/project.

Практическая значимость работы заключается в том, что разработанная пошаговая инструкция может быть использована студентами, начинающими разработчиками и всеми желающими для самостоятельного создания веб-сайтов. В ходе проекта наглядно продемонстрировано, что создание сайта не требует глубоких знаний программирования или больших финансовых затрат: достаточно базового уровня HTML и CSS, бесплатных инструментов разработки и бесплатного хостинга.

Таким образом, гипотеза о том, что создание сайта — процесс, не требующий огромных усилий и навыков, и что его может освоить любой заинтересованный человек, полностью подтвердилась. Поэтапное выполнение работы от планирования до публикации позволило получить готовый результат, который доказывает доступность веб-разработки для начинающих.

СПИСОК ЛИТЕРАТУРЫ

Введение в CSS / MDN Web Docs. – URL: https://developer.mozilla.org/ru/docs/Learn_web_development/Core/Styling_basics/Introduction (дата обращения: 26.03.2026). – Текст : электронный.

1. GitHub Pages. Начало работы / GitHub. – URL: https://pages.github.com (дата обращения: 26.03.2026). – Текст : электронный.

2. GitHub Pages: создание и размещение веб-сайта / Microsoft Learn. – URL: https://learn.microsoft.com/ru-ru/training/modules/create-host-web-sites-github-pages/ (дата обращения: 26.03.2026). – Текст : электронный.

3. Какие дизайн-тренды 2025 года остаются с нами в 2026: практические инсайты Convergent. Часть 2 / Convergent // WORKSPACE. – 2025. – 29 декабря. – URL: https://workspace.ru/blog/kakie-dizayn-trendy-2025-goda-ostayutsya-s-nami-v-2026-prakticheskie-insayty-convergent-chast-2/ (дата обращения: 26.03.2026). – Текст : электронный.

4. МакФедрис, П. Веб-дизайн с нуля: HTML + CSS на практике : пер. с англ. / Пол Макфедрис ; перевел с английского С. Черников. – 2-е изд. – Санкт-Петербург : Питер : Прогресс книга, 2025. – 412 с. – (Библиотека программиста). – ISBN 9785446141289. – Текст : непосредственный.

5. Муслимов, З. О. Практикум по верстке сайтов. Часть 2 : учебное пособие для вузов / З. О. Муслимов. – Санкт-Петербург : Лань, 2026. – 424 с. – ISBN 978-5-507-54531-5. – Текст : непосредственный.

6. Начало работы с CSS / MDN Web Docs. – URL: https://developer.mozilla.org/ru/docs/Learn_web_development/Core/Styling_basics/Getting_started (дата обращения: 26.03.2026). – Текст : электронный.

7. Практикум по верстке сайтов. Часть 1 / Муслимов З. О. – Санкт-Петербург : Лань, 2026. – 448 с. – ISBN 978-5-507-54530-8. – Текст : непосредственный.

8. Тренды дизайна 2025-2026, часть 2. Эмоциональность, честность и живые интерфейсы / Convergent // ADPASS. – 2026. – 21 января. – URL: https://adpass.ru/trendy-dizajna-2026-chast-2-emotsionalnost-chestnost-i-zhivye-interfejsy/ (дата обращения: 26.03.2026). – Текст : электронный.